



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

PROGRAMA DE POSGRADOS

MEJORA CONTINUA

TÍTULO ÁNALISIS DE MEJORA CONTINUA

Autor (es):

Quezada Gallardo, Angelo Diancarlo
aquezadag1@miumg.edu.gt

Profesor

MIXTUN LOPEZ, AURA DENMI

Curso

Diseño de seguridad y CMMI – 2do trimestre - 2026

Guatemala Mayo, Año 2026

INDICE

Introducción.....	3
Análisis del escenario	4
1. Aprender	4
2. Establecer objetivos.....	4
3. Analizar.....	4
4. Desarrollar el plan de acción	4
5. Desplegar mejoras	4
6. Evaluar la capacidad	5
Diagrama de flujo	6
Conclusión.....	7

Introducción

La mejora continua es un pilar fundamental en el desarrollo seguro de software, ya que permite a los equipos evolucionar sus prácticas de seguridad de manera proactiva y sistemática. Dado que las amenazas cibernéticas y las vulnerabilidades cambian constantemente, no basta con implementar controles de seguridad una sola vez; es necesario revisar, ajustar y optimizar periódicamente cada fase del ciclo de vida del desarrollo.

En este contexto, el presente análisis plantea un ciclo de mejora continua estructurado en seis etapas clave: Aprender, Establecer objetivos, Analizar, Desarrollar el plan de acción, Desplegar mejoras y Evaluar la capacidad. Este enfoque, representado mediante un diagrama de flujo, facilita la identificación de brechas de seguridad, la priorización de acciones correctivas y la medición objetiva de los resultados, promoviendo así un proceso de desarrollo cada vez más robusto y resiliente.

Análisis del escenario de Mejora Continua aplicado al proceso de Desarrollo Seguro de Software (DevSecOps)

Análisis del escenario

La mejora continua en el desarrollo seguro de software permite identificar brechas de seguridad, ajustar prácticas y elevar la madurez del equipo. Las fases propuestas se alinean con ciclos como PDCA (Plan-Do-Check-Act) o IDEAL.

1. Aprender

- Revisar incidentes de seguridad previos (postmortem).
- Analizar nuevas vulnerabilidades conocidas (OWASP, CVE).
- Capacitar al equipo en técnicas de seguridad.
- Evaluar resultados de escaneos SAST/DAST.

2. Establecer objetivos

- Reducir tiempo de corrección de vulnerabilidades críticas.
- Aumentar cobertura de pruebas de seguridad.
- Disminuir número de vulnerabilidades en producción.
- Incorporar nuevas herramientas o reglas de análisis.

3. Analizar

- Identificar causas raíz de fallos de seguridad.
- Comparar métricas actuales vs objetivos.
- Evaluar cuellos de botella en el pipeline seguro.
- Priorizar áreas de mejora según riesgo.

4. Desarrollar el plan de acción

- Definir tareas concretas (ej: actualizar reglas SAST, agregar escaneo de dependencias).
- Asignar responsables y plazos.
- Establecer recursos necesarios (herramientas, capacitación).

5. Desplegar mejoras

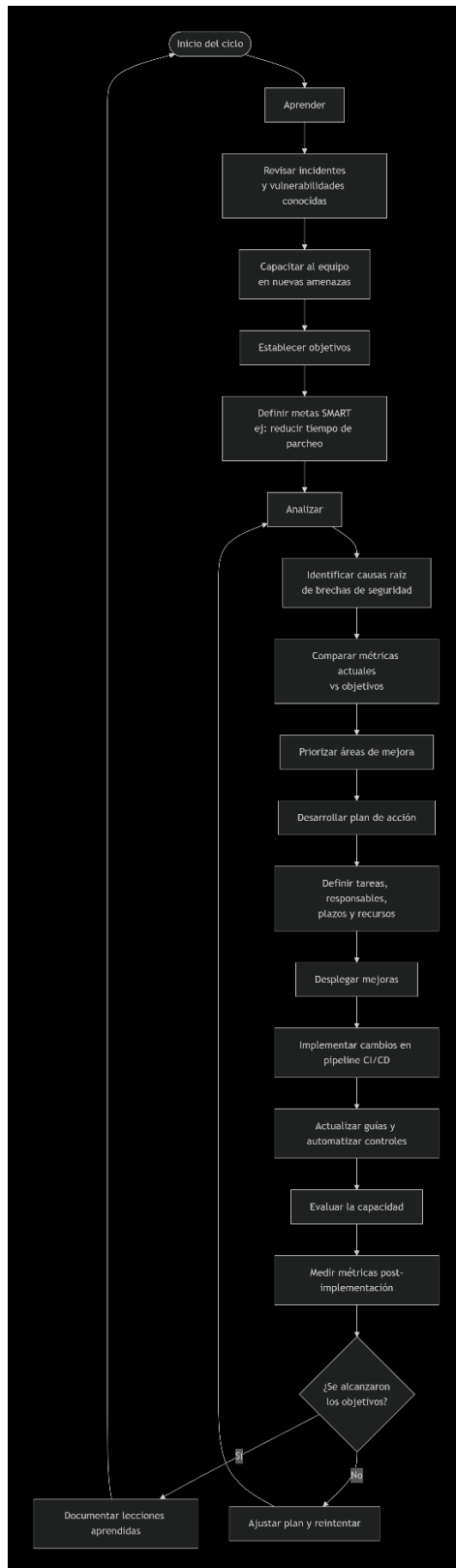
- Implementar cambios en el pipeline CI/CD.
- Actualizar guías de codificación segura.
- Realizar talleres o pair programming en seguridad.

- Automatizar nuevos controles.

6. Evaluar la capacidad

- Medir métricas después de la implementación.
- Verificar si se alcanzaron los objetivos.
- Identificar desviaciones o efectos secundarios.
- Documentar lecciones aprendidas y reiniciar el ciclo en Aprender.

Diagrama de flujo



Conclusión

La implementación de un ciclo de mejora continua en el proceso de desarrollo seguro de software no es un lujo, sino una necesidad en el panorama actual de amenazas cibernéticas. A través del análisis realizado y el diagrama de flujo propuesto, se demuestra que las fases de Aprender, Establecer objetivos, Analizar, Desarrollar el plan de acción, Desplegar mejoras y Evaluar la capacidad conforman un sistema dinámico y autoreflexivo que permite a los equipos de desarrollo evolucionar constantemente.

Entre los principales beneficios de aplicar este modelo se destacan:

- Reducción proactiva de vulnerabilidades antes de que lleguen a producción.
- Optimización del pipeline DevSecOps mediante la automatización de controles.
- Alineación estratégica entre objetivos de negocio y seguridad.
- Cultura de aprendizaje continuo, donde los incidentes se convierten en oportunidades de mejora.

En definitiva, adoptar un enfoque de mejora continua para el desarrollo seguro de software transforma la seguridad de ser un punto de control rígido a convertirse en un habilitador ágil, que se adapta y fortalece con cada iteración.